

Incremental Data Processing with Delta Lake

Assignment Summary — Load, Clean, Merge (Upsert), Validate

Objective

Perform incremental data processing using Delta Lake: load a base dataset into a Delta table, clean it, simulate an incremental batch of new and changed records, apply a MERGE (upsert) operation, and validate the results.

Dataset

A customer-dimension dataset using the same schema as the Kaggle Superstore dataset (CustomerID, CustomerName, Segment, Country, City, State, PostalCode, Region): 32 records in the master file and 12 records in the incremental batch (6 updates to existing customers, 6 brand-new customers).

Pipeline Steps

1. Load

The raw CSV was read into Spark and persisted as a managed Delta table (customer_delta), giving ACID guarantees and a transaction log for every subsequent change.

2. Clean

The base table contained 2 exact duplicate rows and blank values in City, PostalCode, and Segment. Duplicates were removed with dropDuplicates(); blanks were replaced with sensible defaults.

3. Simulate incremental data

A second file, customer_incremental.csv, was created to represent a new incoming batch: 6 existing customers with changed attributes and 6 customers not yet present in the master table.

4. MERGE (upsert)

Delta Lake's MERGE INTO, via DeltaTable.merge(), matched incoming rows to existing rows on CustomerID. Matched rows were updated in place (SCD Type 1); unmatched rows were inserted. An optional SCD Type 2 variant was also implemented, which instead preserves the prior version of a changed row (is_current = false) and appends a new current version, supporting historical queries.

5. Validate

After the merge, the row count grew by exactly the number of new customers, and no duplicate CustomerIDs were found. Delta's history() command also provided a built-in audit trail of every write and merge operation applied to the table.

6. Final output

The final customer_delta table contains the original cleaned records with the 6 changed customers reflecting their latest values, plus the 6 newly inserted customers, with zero duplicate keys.

Validation Results

Metric	Value
Rows after de-duplication (master)	30
Rows in incremental batch	12
Existing customers updated	6
New customers inserted	6
Final row count	36
Duplicate CustomerIDs after merge	0

Why Delta Lake (vs. plain Parquet/CSV)?

- MERGE INTO isn't available on plain files — without it, an upsert means reading the entire table, joining it against the incoming batch, and overwriting everything, which is expensive and unsafe against concurrent readers.
- ACID transactions mean a failed job can't leave the table half-written.
- Time travel and the history() command give a built-in audit trail, which matters for slowly changing dimension tables like this one.

Full runnable code: [notebooks/delta_scd_assignment.ipynb](#) | Dataset schema reference: [Kaggle Superstore dataset \(vivek468/superstore-dataset-final\)](#)